

Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types

Principal Component Analysis (Teoria)

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

Conteúdo

1 Machine Learning: Análise de Componentes Principais (PCA)

A análise de componentes principais (PCA) é um método de aprendizado não supervisionado (*unsupervised learning*) que tenta detectar as direções nas quais os vetores formados por um conjunto de dados variam mais.

Ele primeiro encontra a direção de maior variância e, em seguida, prossegue para descobrir novas direções de maior variância que sejam **ortogonais** àsquelas já encontradas. Assim, para dados de n dimensões, ele retorna (por padrão) n direções ortogonais e as variâncias correspondentes a elas. Estas direções são chamadas de **eixos principais** (*principal axes*), e se projetarmos um ponto do conjunto de dados sobre estes eixos, obtemos os **componentes principais** (*principal components*) de cada eixo.

Para usar outra terminologia, o conjunto de eixos principais forma uma *base* para o espaço vetorial onde os pontos de dados residem, e os componentes principais representam as coordenadas dos pontos neste novo sistema de coordenadas.

A classe PCA na biblioteca `scikit-learn` possui o método `transform`, que converte dados originais para este novo sistema de coordenadas.

1.1 Exemplo de Distribuição Multivariada

Vamos analisar um exemplo de distribuição Gaussiana multivariada submetida ao processo de PCA. Primeiro, nós extraímos aleatoriamente dados de uma distribuição, espalhamos os pontos e visualizamos através de `matplotlib`.

```
import numpy as np
import matplotlib.pyplot as plt
import math
from sklearn.decomposition import PCA

rng = np.random.RandomState(0)
X = rng.randn(2, 400)
scale = np.array([[1, 0], [0, 0.4]]) # Desvios Padroes: 1 e 0.4
rotate = np.array([[1, -1], [1, 1]]) / math.sqrt(2)

transform = np.dot(rotate, scale)
X = np.dot(transform, X).T

# O PCA revelara os eixos principais desse amontoadado
pca = PCA(n_components=2)
pca.fit(X)

print("Eixos principais:\n", pca.components_)
print("Variancia Explicada:\n", pca.explained_variance_)
```

1.1.1 Redução de Dimensionalidade

Você pode ter notado que podemos fornecer ao construtor PCA um parâmetro `n_components`. Esse parâmetro fornece o número de eixos que queremos manter. Se o valor for n , o algoritmo retorna os n componentes com as mais altas variâncias e ignora ativamente os demais. Dessa forma, esse algoritmo também age como uma exímia técnica de **redução de dimensionalidade**.

```
pca = PCA(n_components=1)
pca.fit(X)
Z = pca.transform(X) # Transforma os pontos para caber em uma dimensao
```

A redução pode ser usada, por exemplo, para converter dados hiperdimensionais abstratos para 2 ou 3 dimensões, garantindo que o humano seja capaz de visualizá-los em um gráfico

compreensível. Além disso, essa técnica atua perfeitamente como **pré-processamento**: você pode rodar o PCA antes de injetar os dados em Regressões Lineares e Redes Neurais para reduzir o ruído (fatores que variam pouco) e preservar o cerne dos atributos do conjunto de treinamento.

2 Resumo da Semana 6

- Conhecemos um método de aprendizado supervisionado: a classificação **Naive Bayes**. Usamos *cross validation* para garantir que o modelo não está viciado (*overfitting*).
- Na sessão de *Clustering* (Agrupamento), vimos os exemplos não-supervisionados de **K-Means**, **DBSCAN**, e agrupamento hierárquico, todos baseados na noção de distância entre nós de dados.
- E, por fim, a análise de **PCA** demonstrou uma forma não-supervisionada eficiente para tratar reduções dimensionais filtrando matrizes pela sua capacidade variável explicativa.